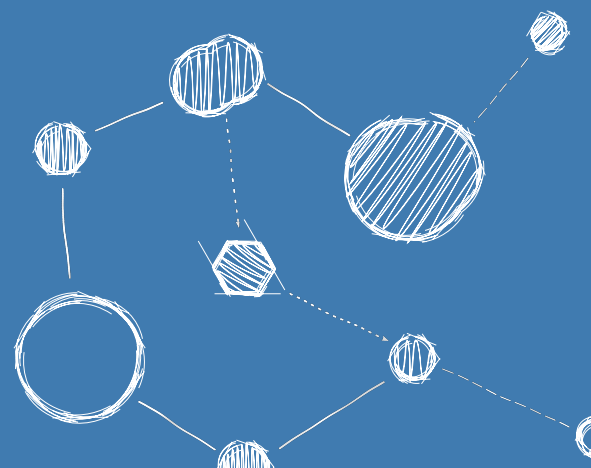


SPW_BRIDGE – A SPACEWIRE TO NETWORK BRIDGE FOR GROUND SUPPORT EQUIPMENT

User Manual



User Manual

Reference: SPW_BRIDGE-UVIE-UM-001

Version: Issue 1.1, September 10, 2026

Prepared by: Armin Luntzer¹

Checked by: Roland Ottensamer¹

Approved by: Franz Kerschbaum¹

¹ Department of Astrophysics, University of Vienna

Copyright ©2026

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Front-Cover, no Logos of the University of Vienna.

Contents

1	Introduction	5
1.1	Purpose of the Document	5
2	Applicable and Reference Documents	6
3	Terms, Definitions and Abbreviated Items	7
3.1	Acronyms	7
3.2	Glossary	7
4	Software Overview	9
4.1	Function and Purpose	9
4.2	Build Requirements	9
4.3	Build System	9
4.4	Source and Support Documentation	10
4.5	Current Platform Configuration	10
4.6	Features Overview	10
5	Structure and Concept	12
5.1	Source Code Organisation	12
5.2	Network Subsystem	12
5.3	SpaceWire Subsystem	13
5.4	RMAP Helper Functions	13
5.5	GRESB Support	13
5.6	ECSS Packet Library	14
5.7	Block Diagram	14
5.8	Data Flow	15
6	Using the Bridge	16
6.1	Synopsis	16
6.2	Operating Modes	16
6.3	SpaceWire Configuration	17
6.4	Interpreting the Network Stream	17
6.5	Monitor Mode	18
6.6	Reading the Monitor Mode Printout	18
6.7	Keyboard Control of the Debug Printout	19
6.8	Reading the PUS Debug Printout	19
6.9	The PUS CRC16 Check	20
6.10	The Pseudo-RMAP Interface	20
6.11	Examples	21

6.11.1	Server Mode with a PCIe Card	21
6.11.2	Link Monitoring	21
6.11.3	Brick MK II Configuration	21
6.11.4	Use in the CHEOPS EGSE	21
6.11.5	Use in the ARIEL Mission	22
6.11.6	GRESB Protocol with grmon	22

Revision History

Revision	Date	Author(s)	Description
1.0	10.09.2026	AL	Initial version
1.1	10.09.2026	AL	add monitor mode bridging two SpaceWire channels with RMAP decoding

1. Introduction

1.1 Purpose of the Document

This document gives an overview on how to build, configure and use the [SpW](#) bridge software. An overview of the structure of the software will be given; detailed information relevant to system programming must however be looked up in the in-line documentation of the source files. The main text describes the generic bridging machinery that the software provides, the command-line interface, and the packet formats exchanged with a peer on the network side.

2. Applicable and Reference Documents

- [1] *ECSS-E-70-41A Space engineering - Ground systems and operations - Telemetry and telecommand packet utilization standard*. ESA Requirements and Standards Division, 2003.
- [2] *ECSS-E-ST-50-12C Space engineering - SpaceWire - Links, nodes, routers and networks*. ESA Requirements and Standards Division, 2008.
- [3] *ECSS-E-ST-50-52C Space engineering - SpaceWire - Remote memory access protocol*. ESA Requirements and Standards Division, 2010.
- [4] *ECSS-E-ST-70-41C Space engineering - Telemetry and telecommand packet utilization*. ESA Requirements and Standards Division, 2016.
- [5] *GR712RC Dual-Core LEON3FT SPARC V8 Processor User's Manual*. 2016.
- [6] *GR740 Quad-Core LEON4FT SPARC V8 Processor Technical Reference Manual*. 2021.
- [7] *GRESB User's Manual*. 2017.
- [8] *GRMON2 User's Manual*. 2018.
- [9] *MSSL-IF-109: Format for the downlink of FEE data packets*. Version Draft A.14. as referenced by the FEE frame counter fields in net.c. Mullard Space Science Laboratory.
- [10] *Space Packet Protocol*. Consultative Committee for Space Data Systems, 2012.

3. Terms, Definitions and Abbreviated Items

3.1 Acronyms

CCSDS	Consultative Committee for Space Data Systems
CRC	Cyclic Redundancy Check
ECSS	European Cooperation for Space Standardization
EGSE	Electrical Ground Support Equipment
EOP	End Of Packet
FEE	Front-End Electronics
GCC	GNU Compiler Collection
PUS	Packet Utilisation Standard
RMAP	Remote Memory Access Protocol
TCP	Transmission Control Protocol
UDP	User Datagram Protocol

3.2 Glossary

Doxygen

Doxygen is a documentation generator for writing software reference documentation.

GR712RC

The GR712RC is a dual-core [LEON3-FT](#) system-on-chip manufactured by Gaisler [\[5\]](#).

GRESB

The GRESB is a SpaceWire/Ethernet bridge by Gaisler that routes [SpaceWire](#) packets to and from TCP sockets over an Ethernet network; it offers three SpaceWire links and six virtual links [\[7\]](#).

LEON3-FT

The LEON3-FT is a fault-tolerant version of the [LEON3](#). Changes to the base version include autonomous error handling, cache locking and different cache replacement strategies.

MSSL

The Mullard Space Science Laboratory is a department of University College London; on CHEOPS it provides the science payload, whose FEE data frames follow MSSL-IF-109 [\[9\]](#).

SpaceWire

SpaceWire is a spacecraft communication network based in part on the IEEE 1355 standard of communications.

SpW

SpW is a common abbreviation of [SpaceWire](#) used throughout this document.

STAR API

The STAR API is the software interface to SpaceWire interface devices such as the Brick MK II, Brick MK III, PCI and PCIe cards; `spw_bridge` uses it for all device, channel and link management.

4. Software Overview

4.1 Function and Purpose

spw_bridge is a software bridge between a [SpaceWire](#) link and a network connection to a workstation. It operates a [SpaceWire](#) interface device through the [STAR API](#) API and relays packets between the [SpW](#) link and a [Transmission Control Protocol \(TCP\)/IP](#) or [User Datagram Protocol \(UDP\)](#) endpoint, in either direction.

On the [SpW](#) side, the software uses the STAR API to select a device and channel, to set the link signalling rate and to submit and receive packets via the STAR transfer-operation machinery. On the network side, it provides a multi-connection server, a single-connection client, or a UDP endpoint. The bridge may optionally interpret the carried data stream: [Packet Utilisation Standard \(PUS\)](#) packets, [Front-End Electronics \(FEE\)](#) data frames, GRESB protocol packets, and a pseudo-[Remote Memory Access Protocol \(RMAP\)](#) command interface are supported and are described below.

The intended use is ground support equipment, e.g. to operate the front-end electronics of an instrument through a [SpaceWire](#)-to-Ethernet path from a workstation, or to debug a [LEON3-FT](#)-based target such as a [GR712RC](#) through the [RMAP](#) protocol.

4.2 Build Requirements

In order to build spw_bridge, make sure you have set up your environment so that the STAR API tools are available. The build currently links against the shared libraries

```
-lstar-api -lrmap_packet_library -lstar_conf_api_generic  
-lstar_conf_api_mk2 -lstar_conf_api_brick_mk2  
-lstar_conf_api_brick_mk3 -lstar_conf_api_pcie_mk2
```

together with `-lpthread`. The matching headers are found under `star-system/inc/star` and are provided by the STAR API installation. You will also need a [GNU Compiler Collection \(GCC\)](#) compiler for your host platform, GNU make and a bash shell, i.e. what you would typically find in a Linux distribution with a basic set of development tools installed. If you want to generate source code documentation, you will also need to install [Doxygen](#).

The [STAR API](#) API, the RMAP packet library and the device configuration libraries are supplied by STAR-Dundee Ltd. as part of the support software of their [SpaceWire](#) devices; obtain them from STAR-Dundee if you are a customer who acquired one of their devices.

4.3 Build System

The build is driven by the plain Makefile in the source root. To build the binary, simply issue

```
$ make
```

on your shell prompt from the top-level `spw_bridge` directory. The build produces an executable named `spw_bridge` in the current directory. To remove the build output, use

```
$ make clean
```

The optional `DEBUG` variable controls the verbosity of the debug output. By default it is set to 1, which disables the per-packet DBG traces. Setting `DEBUG` to 2 or higher enables them:

```
$ make DEBUG=2
```

4.4 Source and Support Documentation

The source files carry Doxygen-style in-line documentation. The [STAR API](#) API header set and its reference documentation are kept below `star-system/`, with the generated [Doxygen](#) HTML under `star-system/doc`. The reference standards drawn upon by the packet handling code are listed in [Chapter 2](#).

4.5 Current Platform Configuration

The bridge is a host-side application and runs on a Linux workstation. The [STAR API](#) device selected by default is device number 0; the device enumeration performed at startup lists the detected devices and their names. A channel number between 0 and 31 may be selected; the default channel is 1. The link signalling rate is set to 10 Mbit/s by default and may be configured between 2 and 400 Mbit/s. Channels, link identifiers and routing paths are under configuration control through the command-line options described in [Chapter 6](#).

4.6 Features Overview

The following gives a concise overview of the currently implemented features:

- Network transport over a single TCP connection (server or client mode) or over UDP datagrams.
- Interpretation of the network byte stream into [PUS](#) packets (ECSS-E-ST-70-41C) with sequence counting and CRC16 verification in both directions.
- Decoded printing of [Consultative Committee for Space Data Systems \(CCSDS\)](#) and PUS-C header fields, the per-packet CRC16 status and the payloads for debugging.
- Interpretation of FEE data frames (with the FEE framing of [MSSL Draft A.14](#)).

- Support for the GRESB protocol, intended for use with the grmon debug monitor for **RMAP** access to a target.
- A pseudo-**RMAP** command service on a dedicated TCP port for lossless read and write access to the **SpW** network.
- Optional stripping of a fixed number of leading bytes of incoming **SpW** packets (e.g. a path header added by the network).
- Optional throttling of **SpW** packet submission by an inserted delay between packets.
- Optional device reset on startup.
- Monitor mode: verbatim copying of packets between two **SpW** channels of the same device, with the network side acting as a passive observer.

5. Structure and Concept

5.1 Source Code Organisation

The bridge consists of a small set of dedicated source files:

<code>spw_bridge.c</code>	main program: option processing, startup and shutdown
<code>spw_bridge.h</code>	shared configuration and subsystem interfaces
<code>net.c</code>	network subsystem: sockets, framing, packet sink
<code>spw.c</code>	SpaceWire subsystem: STAR API device and link handling
<code>kbd.c, kbd.h</code>	keyboard monitor: toggles the PUS debug printout
<code>rmap.c, rmap.h</code>	RMAP command and reply helper functions
<code>gresb.c, gresb.h</code>	GRESB protocol packet helpers
<code>byteorder.h</code>	endianness conversion macros
<code>debug.h</code>	debug output control
<code>ecss/</code>	CCSDS, ECSS (PUS) and CRC packet library

The two subsystems talk to each other through the `bridge_cfg` structure: the network code installs a packet sink callback (`cfg->pkt_sink = net_pkt_sink`) that the `SpW` receive loop invokes for every complete packet received on the link.

5.2 Network Subsystem

The network subsystem maintains a per-service `fds` set and multiplexes its connections with `select()`. Two services exist: the *data* service carries the forwarded `SpW` packets, and the *rmap* service implements the pseudo-RMAP interface (see Section 6.10). Each service has its own accept and poll thread; the RMAP service is always started as a listening server, even when the data path runs in client mode. A peer that stays silent for more than 30 seconds is dropped.

Depending on the selected mode, a received network byte stream is framed as follows:

- *raw*: one `recv()` call defines one `SpW` packet. Note that back-to-back packets sent on a stream socket may be merged into a single receive, and a partially delivered packet may be split, so a higher-level framing protocol is normally required for reliable transport.
- *PUS* (-P): a six-byte CCSDS primary header is read first; the packet length field plus one octet plus the six-byte header defines the packet boundary. The sequence counter in bytes 2 and 3 is checked against the running sequence in both directions and mismatches are reported on `stderr`.
- *FEE* (-F): a ten-byte FEE data header is expected; if its protocol identifier is `0xF0`, the big-endian data length field plus the header defines the packet boundary. A different

protocol identifier defers to raw framing.

- *GRESB* (-G): a four-byte host-to-GRESB header (protocol byte plus three-byte big-endian size) defines the packet boundary.

In the other direction, every packet received on the [SpW](#) link is passed to `net_pkt_sink`, which forwards it to all connected data clients. In GRESB mode the [SpW](#) packet is wrapped into a host-to-GRESB data packet before transmission.

5.3 SpaceWire Subsystem

The [SpW](#) subsystem selects the STAR device and channel, optionally resets the device, sets the link signalling rate through `CFG_setTransmitSignallingRate`, opens the channel and creates the routing path with `STAR_createAddress`. The link is then forced into the running state and the measured receive rate is reported.

A dedicated polling thread is started which repeatedly submits a single receive transfer operation (`STAR_createRxOperation` with EOP terminated packets) and waits for its completion. On completion the received packet is handed to the network sink. Transmission uses `STAR_createPacket` plus a transmit transfer operation, with an optional throttle delay after each packet. Reads and writes of registers on remote targets are implemented with the STAR RMAP packet library, see below.

In monitor mode (Section [6.5](#)) two channels of the device are opened instead of one, each polled by its own thread; received packets are copied to the other channel without a routing header.

5.4 RMAP Helper Functions

The RMAP helper sources provide verification, CRC-8 computation, and parsing of RMAP packets, including a table of the command codes and status codes of [ECSS](#) ECSS-E-ST-50-52C. The bridge uses these helpers to parse RMAP packets for logging and to decode replies received from targets on the [SpW](#) link.

5.5 GRESB Support

The GRESB is a [SpaceWire](#)-to-Ethernet bridge intended for ground equipment (see the [GRESB](#) glossary entry and [\[7\]](#)). The bridge implements its host protocol: a data transfer consists of a one-byte protocol identifier followed by a three-byte big-endian size field and the raw [SpW](#) data. The reverse direction carries a header with reserved, truncated and [End Of Packet](#)

(EOP)-error bits followed by the three-byte size and the data.

The GRESB uses a base port number of 3000 with two ports per virtual link: the even port is the transmit port and the odd port is the receive port. Up to five virtual links are supported. This mode is intended to be used with the grmon debug monitor for [RMAP](#) access (see Section 6.11.6).

5.6 ECSS Packet Library

The `ecss/` directory contains a self-contained packet library that follows the CCSDS Space Packet Protocol [10] and the PUS [4]:

- `ccsds_pkt.c/h` – field accessors and setters for the six-byte CCSDS primary header (version, type, secondary header flag, APID, sequence flags and count, data length).
- `ecss_pkt.c/h` – PUS-A and PUS-C secondary header handling, verification support, source and destination identifiers, timestamps, and a service/sub-service model. The active PUS version is configured in `ecss_pkt_cfg.h`.
- `ecss_pkt_cfg.h` – configuration and service type / sub-service definitions (ST_VER, ST_HK, ST_EVT, ST_Mem, ST_Time, and so on). The current configuration tracks PUS-C as the baseline and defines a maximum packet size of 4096 bytes.
- `crc16.c/h` – CRC16-CCITT-FALSE with a slice-by-8 lookup table and a benchmark suite.
- `bitman.c/h` – bit-level field extraction used by the packet library.

The `pus_debug_print()` function uses this library to print decoded CCSDS and PUS-C headers and payloads for both directions when enabled with `-D` and `-P`. The CRC16 check on the data path is performed with `crc16_buf()` from the same library (see Section 6.9).

5.7 Block Diagram

Figure 5.1 shows the connectivity design of the bridge. Ground support equipment, a grmon debug monitor or a GRESB device exchange data with the bridge over Ethernet (TCP/IP or UDP). The network side of the bridge frames the received byte streams according to the selected packet mode, the relaying core hands the packets to the SpW side, which submits them through a STAR API API device onto the SpaceWire link, and the same path is used in reverse for packets received from the link.

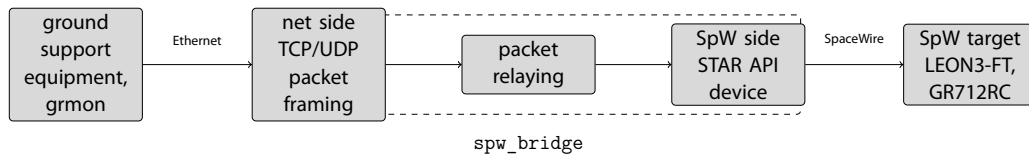


Figure 5.1: Block diagram of the `spw_bridge` connectivity

5.8 Data Flow

In the *network to SpaceWire* direction, a poll thread waits for data on the data service, frames it according to the selected mode, logs it, and hands it to `spw_send_packet` which transmits it on the [SpW](#) link (see Figure 5.1). The `rmap` service instead parses a pseudo-[RMAP](#) command header (see Section 6.10) and issues the corresponding read or write command.

In the *SpaceWire to network* direction, the [SpW](#) polling thread receives a packet and calls `net_pkt_sink`. Depending on the configuration, the packet is logged, its PUS sequence is checked, and any RMAP replies (identified by the RMAP protocol identifier byte after the skipped header) are forwarded to the RMAP service clients. The data is then forwarded to the data service clients, optionally wrapped in a GRESB packet.

6. Using the Bridge

6.1 Synopsis

```
$ ./spw_bridge [OPTIONS]
```

The bridge runs until it is interrupted; SIGINT, SIGTERM and SIGHUP terminate the program and shut down the device cleanly. The full option set is:

- i DEVNUM **SpW** device id to use (default 0)
- c CHANNEL **SpW** channel to use (default 1)
- n PATH routing path, colon-separated hex bytes, one byte per node
- p PORT local data port number (default 1234)
- s ADDRESS local source address (default 0.0.0.0)
- r HOST:PORT client mode: address and port of the remote target
- u use UDP instead of TCP
- d NUM number of header bytes to drop from incoming **SpW** packets (default 0)
- t USEC throttle: delay in microseconds inserted before each **SpW** packet (default 0)
- S MBPS link speed in Mbit/s (default 10, valid range 2–400)
- L LINKID id of the link whose speed is set (default: equal to the channel)
- P parse the network byte stream for **PUS** packets
- D print decoded PUS-C headers and payloads (requires -P)
- C disable the PUS CRC16 check (see Section 6.9)
- F parse the network byte stream for FEE data packets
- R [PORT] enable the pseudo-**RMAP** service on PORT (default 2345)
- M C1:C2 monitor mode: bridge the given **SpW** channels, copying packets verbatim between them
- E decode **RMAP** packets in the debug printout (requires -M and -D)
- G use the GRESB protocol for the network exchange
- X perform a device reset before opening the channel
- h, --help print the usage message and exit

A routing path given with -n is a colon-separated list of hex byte values, one per node (e.g. -n 01:24:00:00:00); up to 256 nodes are accepted and out-of-range values are truncated to 8 bits.

6.2 Operating Modes

The bridge supports three modes of operation:

- *server mode* (default): the bridge listens on the configured address and port for an arbitrary number of TCP connections. Every connected peer receives the **SpW** traffic, and data from any peer is transmitted on the **SpW** link.

- *client mode* (`-r`): the bridge connects to the specified `HOST:PORT` and operates a single connection. If the link is lost, the bridge terminates.
- *UDP mode* (`-u`): the bridge binds a UDP socket and sends one datagram per [SpW](#) packet. Peers are registered as they are seen, i.e. the first datagram from a host registers it as a recipient.

6.3 SpaceWire Configuration

At startup the bridge lists the detected [STAR API](#) devices and their names. The device is selected with `-i` (default 0). The channel is selected with `-c`; a non-existent channel is rejected and the available channels are printed. The link signalling rate is set with `-S` to a value between 2 and 400 Mbit/s; the actual, measured receive rate is reported during link start. `-L` selects the link whose speed is set; on the Brick MK II the transmit signalling rate of the second port is configured in this way. `-X` resets the device before the channel is opened; note that this affects other ports of the device as well.

6.4 Interpreting the Network Stream

By default the network byte stream is forwarded without framing (raw mode); please note the packet merging and splitting caveat described in [Section 5.2](#). For reliable transport of packet-oriented traffic, use one of the packet modes:

- `-P` frames the stream as [PUS](#) packets, checks the CCSDS sequence counter in both directions and verifies the PUS CRC16 (see [Section 6.9](#)). Sequence mismatches are reported to `stderr`, together with the expected and received counter values.
- `-F` frames the stream as FEE data packets (protocol identifier `0xF0`). All other packets fall back to raw framing.
- `-G` frames the stream as GRESB protocol packets (see [Section 6.11.6](#)).

`-D` additionally prints a decoded CCSDS and PUS-C header and payload dump of every packet passing through the bridge, in both directions. It requires `-P`; once enabled, the printout can also be switched on and off from the keyboard while the bridge is running (see [Section 6.7](#)).

6.5 Monitor Mode

Monitor mode turns the bridge into a passive tap between two **SpW** channels of the same device. It is enabled with `-M CHANNEL1:CHANNEL2`; the two colon-separated channel numbers select the physical **SpW** ports to be bridged. The two channels must differ.

In this mode every packet received on one channel is copied verbatim to the other, without adding or stripping a routing header and without any interpretation on the forwarding path. The bridge neither inserts routing bytes nor drops packet headers, and the forwarded **SpW** transactions remain atomic. The network side works as before, but it only observes: all data received from the network is discarded, and the connected clients receive a mirror of the copied **SpW** traffic in both directions. The channel selection `-c` and the header-drop option `-d` are ignored; the routing path given with `-n` is not applied to the copied packets. The link signalling rate set with `-S` is applied to both channels.

With `-D`, the debug printout shows the direction of the copied packet as `SPW[A] -> SPW[B]` instead of the network directions, followed by a hex dump of the raw packet payload. With the additional `-E` option the packets are decoded as **RMAP** and a header line is printed in front of the payload dump (see Section 6.6). A packet that cannot be parsed as **RMAP** is reported as such. No **PUS** interpretation is performed on the monitor path, and packets are never modified by the decoding.

6.6 Reading the Monitor Mode Printout

In monitor mode, the debug printout of every copied packet is preceded by a time stamp and the direction of travel, using the actual channel numbers of the bridge (e.g. `SPW[1] -> SPW[2]` for a packet received on channel 1 and copied to channel 2). Without `-E` the packet is shown as a raw hex dump of its payload:

```
[2026-09-10 10:20:30.512] SPW[1]->SPW[2]
  payload (18 bytes):
    00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
    10 11
```

With `-E`, a decoded **RMAP** header line is printed in front of the payload dump; a write command and the matching read reply, for example, look like

```
[2026-09-10 10:20:30.512] SPW[1]->SPW[2]
  RMAP: CMD WRITE dst=0xfe key=0xab src=0x30 tr_id=1 addr=0x00001000 data_len=2
        hdr_crc=0x1b data_crc=0x4a
  payload (19 bytes):
    00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
    10 11 12

[2026-09-10 10:20:32.013] SPW[2]->SPW[1]
  RMAP: REPLY READ dst=0x30 key=0x00 src=0xfe tr_id=1 addr=0x00001000 data_len=5
        hdr_crc=0x77 data_crc=0x2c
  payload (22 bytes):
    00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
    10 11 12 13 14 15
```

The printed fields are

SPW[A]→SPW[B]	direction of travel: the channel the packet was received on, and the channel it is copied to
CMD, REPLY	whether the packet is an RMAP command or a reply
WRITE, READ	the operation of an RMAP command
dst, src	target and initiator logical addresses
key	command authorisation key (status field in replies)
tr_id	transaction identifier
addr	(first) data address
data_len	length of the RMAP data field in octets
hdr_crc, data_crc	the header and data CRC-8 values carried by the packet

A packet that cannot be parsed as **RMAP** is announced with a not an **RMAP** packet line and only its payload is dumped. The payload itself is formatted as in the PUS printout (Section 6.8): hex octets, sixteen per line, with an empty line closing the dump.

6.7 Keyboard Control of the Debug Printout

When `-D` is active, the decoded debug printout can be toggled from the terminal while the bridge is running: pressing `d` or `D` turns it on or off, and each change is announced on the terminal with a `PUS debug enabled` or `PUS debug disabled` message. This allows a quick look at the PUS traffic without restarting the bridge or resubmitting the command line. The keyboard control is handled by a dedicated monitor thread and requires `stdin` to be connected to a terminal; if the bridge runs in the background or with redirected input, the toggle is not available.

6.8 Reading the PUS Debug Printout

With `-D` and `-P`, every packet is preceded by a time stamp and the direction of travel, followed by a decoded CCSDS and PUS-C header dump and a hex dump of the payload. An example for a telecommand (TC) with the debug output enabled is

```
[2026-09-10 10:20:30.512] NET->SPW
CCSDS: type=TC APID=0x101 seq_flags=3 seq_count=42 data_len=35
PUS-C: ST=5 SST=1 source_id=0x0000 ACK=0b0110
CRC=OK
payload (30 bytes):
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
10 11 12 13 14 15 16 17 18 19
```

and for the corresponding telemetry (TM) return traffic

```
[2026-09-10 10:20:31.048] SPW->NET
CCSDS: type=TM APID=0x088 seq_flags=3 seq_count=7 data_len=31
PUS-C: ST=1 SST=33 MTC=4 DEST_ID=0x0005 T_REF_STS=0010 TS=5c:00:12:34:00:42
CRC=OK
payload (18 bytes):
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
10 11
```

The printed fields are

APID	application process identifier of the CCSDS primary header
type	TM or TC
seq_flags, seq_count	CCSDS sequence flags and sequence count (decimal)
data_len	CCSDS data length in octets
ST, SST	PUS service type and subservice type
source_id	TC source identifier
ACK	TC acknowledgement flags, printed as four bits
MTC	TM message type counter
DEST_ID	TM destination identifier
T_REF_STS	TM time reference status, printed as four bits
TS	TM time, six-octet PUS-C time of day
CRC	result of the CRC16 check (OK or NOK, see Section 6.9)

The payload is dumped as hex octets, sixteen per line, with an empty line closing the dump. A payload that ends with a partial line is terminated properly, so every packet dump is followed by a blank line.

In monitor mode (Section 6.5) the dump shows the raw copied packet; with `-E` an [RMAP](#) header line (command or reply, read or write, addresses, transaction identifier, data length and the CRCs) is printed in its place.

6.9 The PUS CRC16 Check

Every [PUS](#) packet carries a CRC16 field, as mandated by [ECSS-E-ST-70-41C](#). With `-P`, the bridge verifies this field on every packet in both directions and reports a failed check on `stderr`. The result is also printed in the debug dump as a `CRC=OK` or `CRC=NOK` line directly before the payload. The bridge never drops a packet with a failed check, it only reports it; it is a mediator on the data path. `-C` disables the check, which also suppresses the corresponding printout line. Disable it only when a mission-specific tailoring of the stream omits the CRC16 field.

6.10 The Pseudo-RMAP Interface

With `-R` (optionally followed by a port number, default 2345) the bridge starts a second TCP server that implements a simplified [RMAP](#) command interface. Each command has the form

```
<spw addr><op>:<addr>:<size>:<data>
```

encoded as a ten-byte header, followed by the write payload for write operations:

- byte 0 target [SpW](#) logical address
- byte 1 operation: 0 = read, non-zero = write
- bytes 2–5 32-bit starting address, big-endian
- bytes 6–9 data size, big-endian
- bytes 10... write payload (write operations only)

A read command issues a read of 5 bytes at the given address; the size field is currently ignored for reads. A write command writes the given payload with address increment. The RMAP commands are assembled with the STAR RMAP packet library, using a source logical address of

0x30 and the command key 0xab without an extended address. The RMAP replies produced by a target (returned to 0x30) are detected on the [SpW](#) link by their RMAP protocol identifier and forwarded to the connected RMAP clients in the same ten-byte-plus-data format.

For example, using netcat, a read of 5 bytes at address 0x1000 of a target with logical address 0xfe is issued with

```
$ printf '\xfe\x00\x00\x00\x10\x00\x00\x00\x00' |
nc localhost 2345
```

and a two-byte write of 0xde 0xad to the same address with

```
$ printf '\xfe\x01\x00\x00\x10\x00\x00\x00\x02\xde\xad' |
nc localhost 2345
```

6.11 Examples

6.11.1 Server Mode with a PCIe Card

Transmit and receive on channel 2 of the first detected device:

```
$ ./spw_bridge -c 2
```

6.11.2 Link Monitoring

To observe the traffic between two [SpW](#) targets connected to channels 1 and 2 of the same device, copy the packets verbatim between the channels and decode them as [RMAP](#):

```
$ ./spw_bridge -M 1:2 -D -E
```

The network clients see a mirror of both directions of the copied traffic.

6.11.3 Brick MK II Configuration

The Brick MK II channel is always 1. Configure the signalling rate of link 2 and send packets via the route with id 2 to the remote [SpW](#) device with node address 0x14:

```
$ ./spw_bridge -c 1 -n 2:14 -L 2
```

The same configuration in client mode, connecting to localhost:1234:

```
$ ./spw_bridge -c 1 -n 2:14 -L 2 -r localhost:1234
```

6.11.4 Use in the CHEOPS EGSE

With the real data processing unit (DPU) connected to the [Electrical Ground Support Equipment \(EGSE\)](#) Brick, use link 1 of the Brick as follows:

```
$ ./spw_bridge -c 1 -n 01:24:00:00:00 -d 4 -L 1 -p 5573 -P
```

and link 2 of the Brick accordingly:

```
$ ./spw_bridge -c 1 -n 02:24:00:00:00 -d 4 -L 2 -p 5573 -P
```

The first byte of the route is consumed by the Brick as the link number, the next byte is the address of the DPU (0x24) and the remaining three bytes are consumed by the DPU. The `-d 4` option drops the four-byte **SpW** address header that the real DPU attaches to its packets. Traffic is sent to local port 5573 in **PUS** framing mode.

6.11.5 Use in the ARIEL Mission

For the ARIEL mission, using a PCIe MK II card, the bridge is typically started as follows:

```
$ ./spw_bridge -i0 -P -n52:02:00:00 -p5573 -S60 -d4 -c3
```

This selects device number 0 (`-i 0`), parses the network byte stream as **PUS** packets (`-P`), uses the routing path 52:02:00:00 (`-n`), listens on local port 5573 (`-p 5573`), sets the link signalling rate to 60 Mbit/s (`-S 60`), drops the four-byte header of incoming **SpW** packets (`-d 4`) and opens **SpW** channel 3 (`-c 3`). The channel flag selects the physical **SpW** port of the card and may need adjustment depending on which port is wired to the target.

6.11.6 GRESB Protocol with grmon

For **RMAP** access through the grmon debug monitor, connect the target **SpW** link to a **SpW** port of the used device that supports **RMAP** (on the **GR712RC** only link 0 or link 1 support **RMAP**). Start the bridge with the GRESB option and set the link speed to 10 Mbit/s:

```
$ ./spw_bridge -c 1 -S 10 -G
```

The GRESB uses a base port number of 3000 with two ports per virtual link (the even port for transmit, the odd port for receive). Verify that these ports are forwarded to the data port of the bridge, e.g. with socat:

```
$ socat tcp-l:3000,fork,reuseaddr tcp:127.0.0.1:1234
$ socat tcp-l:3001,fork,reuseaddr tcp:127.0.0.1:1234
```

You may now start grmon against the machine that runs the bridge:

```
$ grmon -gresb 127.0.0.1
```

There is a caveat: the GRSPW2 **SpW** port of the target may not be initialised if no boot ROM is present. For the **GR712RC** you have to connect through the JTAG or another debug interface first and initialise the core, e.g. GRSPW0:

```
grmon> wmem 0x80100800 0xA0010006
grmon> wmem 0x8010080C 0x00000909
```

For details refer to the GRMON2 manual [8].